

Anime Face Generation with VAE

Max Raxy

Udacity Institute of AI & Technology

Capstone-5: Generative AI Applications

August 31, 2026

Anime Face Generation with VAE

Contents

Overview	4
Dataset Description	4
Model Design and Training Approach	5
Dataset Transformation	5
Model Block Diagram	6
VAE Loss Function	8
Optimizer	9
Output Evaluation and Interpretation	10
Learning Curves	10
Image Loss Distributions	11
Latent Vectors Distribution	12
Top-4 Images with Best and Worst Reconstruction	14
Sample Images with their Reconstructed versions	15
Sample Generated Images From Noise Vector	15
Discussion of Generated Outputs: Quality, Variability, and Limitations	17
Ethical Considerations and Responsible Use	17
Limitations and Future Improvements	18
References	18

List of Figures

1	Width and Height Distributions.	4
2	Sample original images resized to 64×64	5
3	Dataset Transformation Pipeline.	6
4	Auto-Encoder(AE) vs Variational Auto-Encoder(VAE).	7
5	Convolutional-Block(ConvBlock) and De-Convolutional-Block(DeConvBlock).	7
6	Encoder Block, Decoder Block and Complete VAE Model.	8
7	Training Learning Curves.	10
8	Left: mode averaging (blurriness), Right: best model	11
9	Image Loss Distributions	12
10	Sample Latent Vectors Distributions.	13
11	Sample Latent Vectors Q-Q Plot.	13
12	Latent Vectors Correlation Matrix.	14
13	Top-4 Images with Best and Worst Reconstruction.	15
14	Sample Random Images with Their Reconstruction.	16
15	Sample Generated Images From Noise Vector.	16

Overview

We are using **Anime Face Dataset** from **Kaggle** (splcher, 2019; Chao, 2019). The dataset contains 63565 anime-face images, and the goal is to use **Variational Auto-Encoder (VAE)**, (Kingma & Welling, 2019; Kingma & Welling, 2022) to generate anime faces.

Dataset Description

Dataset contains anime faces with aspect-ratio 1¹. The height and width varies between 25 to 220 (Fig. 1)

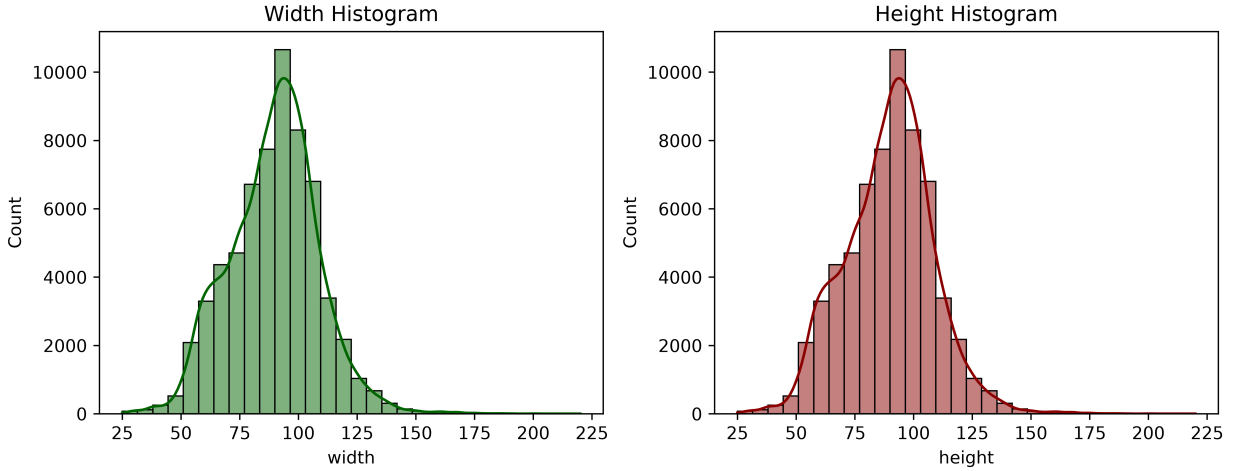


Figure 1

Width and Height Distributions.

We decided to resize all images to 64×64 for this project, this size still has good quality, and proper for training on home-PC with limited resources. Sample of original images resized to 64×64 are shown in Fig. 2

The dataset with 63565 images is large enough for training a VAE model, and we split it to training: 50853 (80%), validation: 6356 (10%) and test: 6356 (10%).

¹ Aspect-ratio has slight deviation from 1 with mean=1.000003, standard deviation =0.000171



Figure 2

Sample original images resized to 64×64 .

Model Design and Training Approach

Dataset Transformation

We used a simple transformation mainly to resize the images, and only used extra random horizontal flip in training transformation that make sense to have it in the face images, no other transformation used to avoid injecting abnormal face into the model (Fig. 3)

Using normalization at last stage of transformation depends on other factors, for example if we want to use Binary-Cross-Entropy (BCE) (Wikipedia contributors, 2026c) as a reconstruction loss function, we need to use *Sigmoid* as activation function (Wikipedia contributors, 2026a) in last layer of our model, then this requires that the dataset map to $[0, 1]$ range, and no normalization required. However if we are going to use other loss function like mean-square-error we can use *Tanh* as activation function and use normalization at last stage and map dataset to $[-1, 1]$ as shown in Fig. 3.

As we can see, there is one optional normalization section in transformation and it

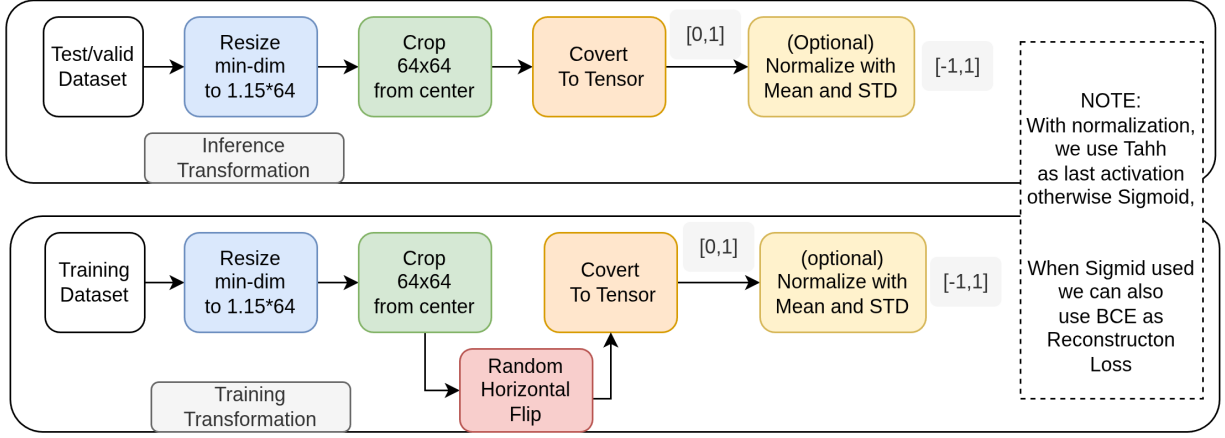


Figure 3

Dataset Transformation Pipeline.

can affect other parameters in the overall system, and we will see similar optional parameters in other parts of our system in this project. We used many configuration options collected in a *DataConfig* class in the notebook, to test and try many of these variations, and finally pick one for final project delivery, for example we did not use normalization in our dataset transformation since we see more promising result when using BCE as reconstruction loss with Sigmoid-activation at last layer of the model.

Model Block Diagram

An Auto-Encoder is an unsupervised neural network trained to compress input data into a lower-dimensional bottleneck representation and then reconstruct it as accurately as possible (Bank et al., 2020), While A Variational Auto-Encoder (VAE) (Kingma & Welling, 2019) is a generative deep learning model that maps input data to a probabilistic latent space (Wikipedia contributors, 2026f), enabling both efficient data compression and the generation of realistic new samples (Fig. 4).

Since we are dealing with image as an input to encoder that suppose to compress the data into smaller latent dimension (or probabilistic latent dimension), we can use usual Convolutional Neural Network (CNN) (Wikipedia contributors, 2026b), to extract information from input image, while we can use stride=2 in CNN block to reduce the

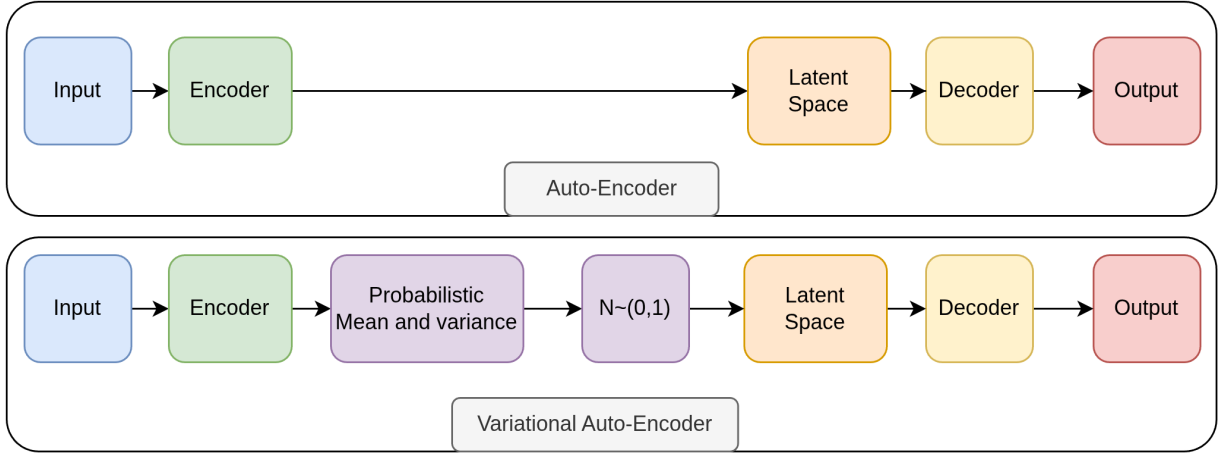


Figure 4

Auto-Encoder(AE) vs Variational Auto-Encoder(VAE).

spatial dimension by 2 (toward having lower-dimension), and similar to famous VGG-model (Simonyan & Zisserman, 2015), we double the channel in next block to extract more abstract semantic information using more filters/channels. The the decoder needs to do reverse operations. We organized our model definition, into Convolutional, De-Convolutional, Encode, and Decoder blocks as shown in Fig. 5 and Fig. 6.

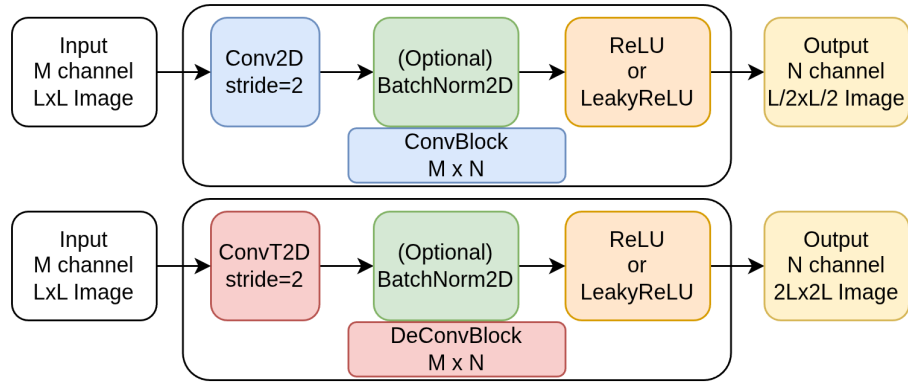


Figure 5

Convolutional-Block(ConvBlock) and De-Convolutional-Block(DeConvBlock).

Fig. 6 shows 4 layers of CovBlock, 64 filters in first CovBlock, and 128 as the

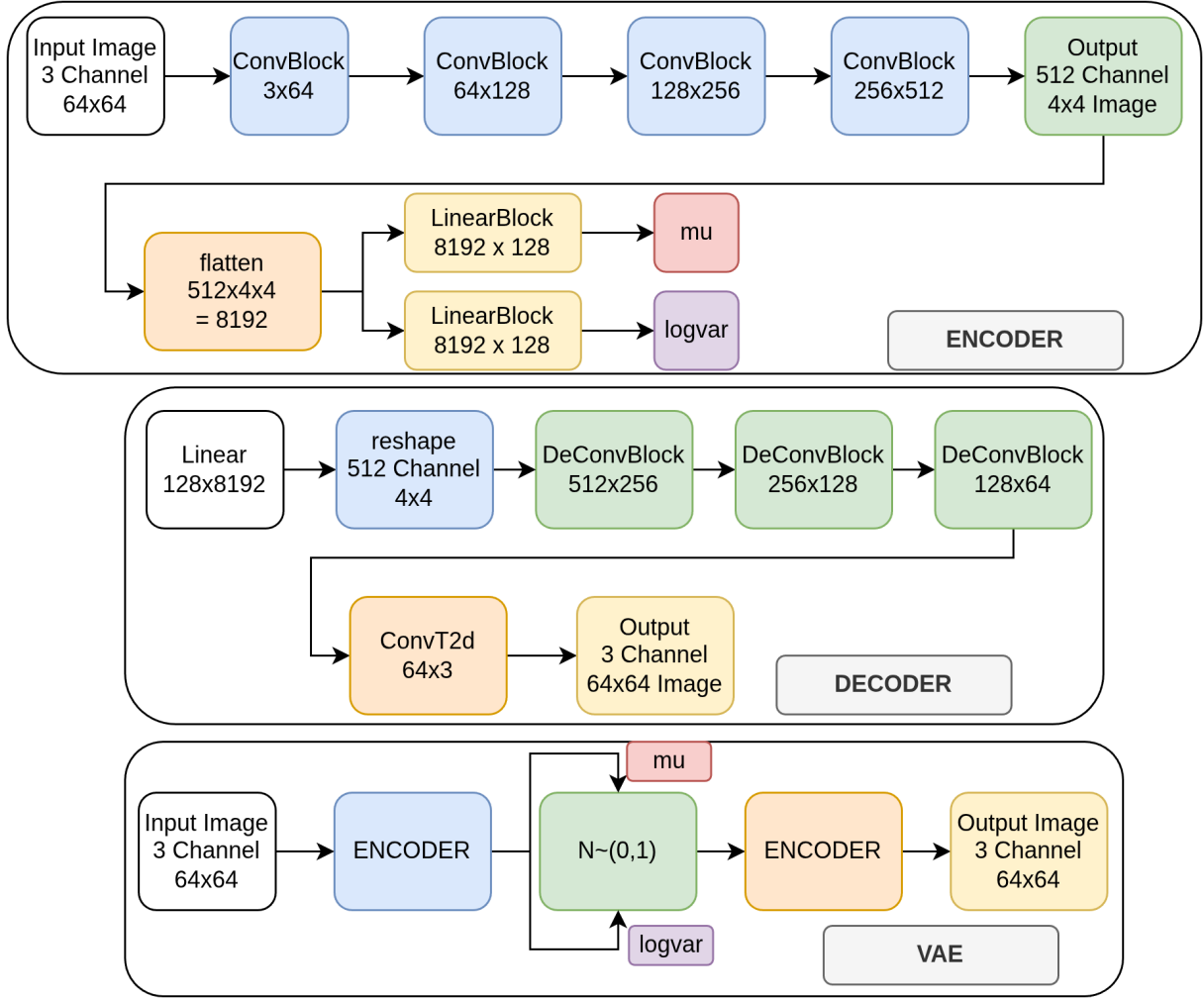


Figure 6

Encoder Block, Decoder Block and Complete VAE Model.

dimension for latent-space which are used in our final model².

VAE Loss Function

In VAEs, the overall objective (Kingma & Welling, 2019) is to minimize the negative Evidence Lower Bound (ELBO) (Wikipedia contributors, 2026d):

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{rec}} + \beta \cdot \mathcal{L}_{\text{KLD}}$$

² These parameters are part of *DataConfig* class in the notebook, and can be changed.

Here, the reconstruction loss measures how accurately the decoder reconstructs the input (e.g. using mean squared error), while the Kullback-Leibler Divergence (KLD) (Wikipedia contributors, 2026e) forces the latent space representation to approximate a standard normal distribution $\mathcal{N}(0, \mathbf{I})$.

For reconstruction loss we can try Mean Square Error (MSE), Mean Absolute Error (MAE) , a combination of MSE and MAE, or Binary-Cross-Entropy (BCE). We have briefly tried these options and we find out BCE produce better (sharper) generated samples, therefor we focus on optimizing our work based on BCE, other option may need more tuning or adjusting other parameters that can be considered as future works.

β is KLD-weight in total-loss, and need to be adjust properly. After many trials we fond out that in our setup, we have to set β to very small values $[0.00025, 0.00035]$ then we start from 0.00025 and slowly increase it to 0.00035 during 50 epochs, to make sure at the beginning it's not affecting the model training for reconstruction, instead of using linear increase we use the following formula that stay low, and ramp up with delay:

$$\beta(epoch) = 0.00025 + 0.0001 \times \left(\frac{epoc}{50}\right)^3$$

After we found out that our best model (least total-loss on validation dataset) happen on epoch=27, for demonstration purpose we aggressively increase β after epoch 30 to reduce the KLD-loss and show the failure mode on VAE where all the image become blurry , washed-out or kind of similar to each other (mode averaging (blurriness)).

Optimizer

We used Adam Optimizer (PyTorch Foundation, 2026a), which delivering fast convergence and strong performance across a wide range of architectures with minimal hyper-parameter tuning. we also include simple ExponentialLR (PyTorch Foundation, 2026b) to decrease the learning rate per epoch as the training progress.

Output Evaluation and Interpretation

Learning Curves

Fig. 7 shows training learning curves.

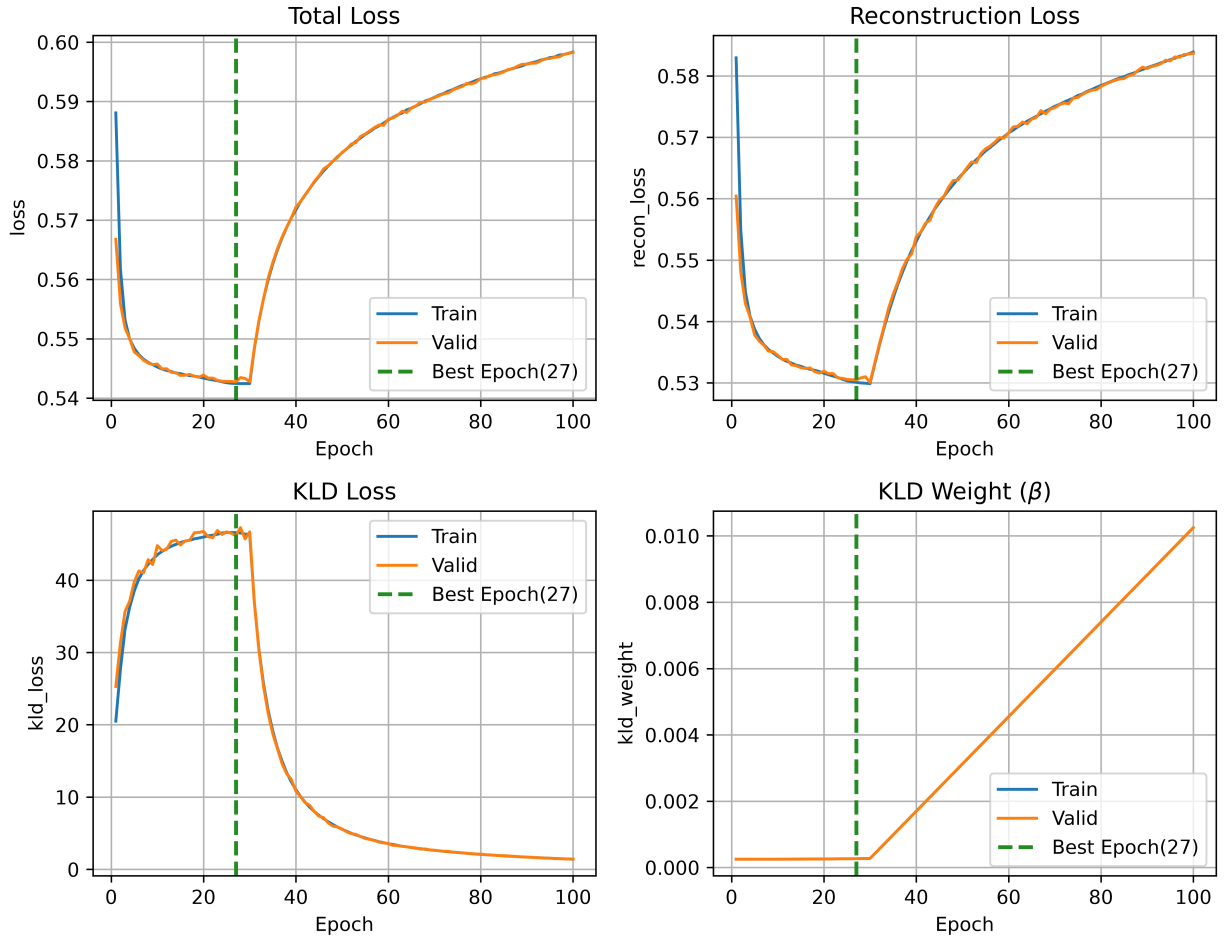


Figure 7

Training Learning Curves.

- At epoch 27 we have the best total validation loss
- We keep KLD-weight low at initial epochs and after the model trained and established its reconstruction capability we keep increasing KLD-weight, at some point around epoch 27, we can see KLD-loss start dropping, and forcing the model to pay more attention to KLD, cause the total construction loss increase.

- Overall epoch 27 is the sweet-spot, where we have best validation total-loss, and KLD is limited and not going up anymore.
- Pushing the KLD-weight higher, reduces the KLD-loss further, but degrades generation diversity and causes mode averaging (blurriness), making the generated anime faces overly uniform.
- For demonstration purpose we intentionally increase KLD-weight to show that we can keep decreasing KLD-loss, but it will affect the reconstruction loss (and consequently total-loss), and also the generated images can be affected (mode averaging (blurriness) Fig. 8).

Generated during training with last-epoch model Generated during training with best model

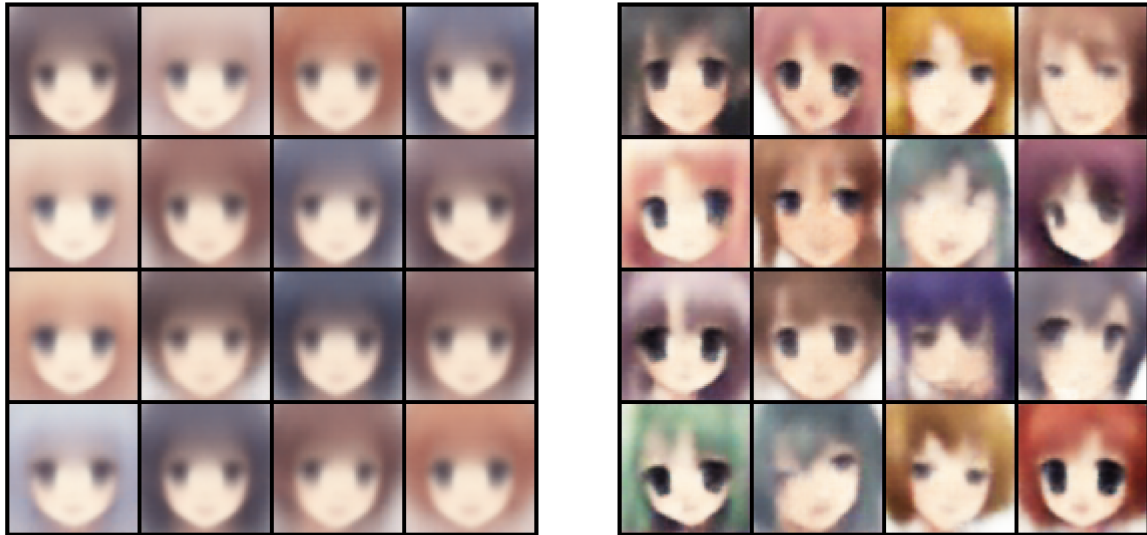


Figure 8

Left: mode averaging (blurriness), Right: best model

Image Loss Distributions

Fig. 9 shows the loss distributions for all images using the best model.

- During training when we calculate the loss, it will be average per batch or per epoch, however in Fig. 9 we did not use any averaging, and the loss value is the loss per

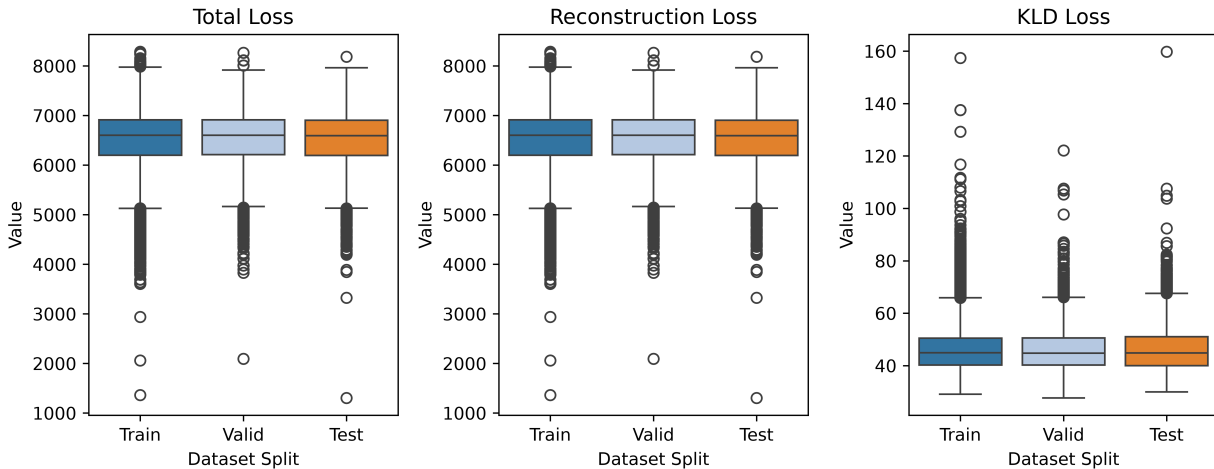


Figure 9

Image Loss Distributions

image, which will help us to investigate images with extreme low or high losses as we will see in next sections.

- While there are some outliers, Overall the average (median) for Total-Loss, Reconstruction-Loss, and KLD-Loss, are similar per Split (Train/Validation/Test). Also the overall distribution (e.g. IQR range) are similar.
- Therefore investigation images on test dataset can be a good representative of overall dataset.

Latent Vectors Distribution

Ideally, the latent vectors expected to have normal distribution, and uncorrelated to each-other (Ideally correlation is an Identity matrix). Fig. 10 shows the distribution of few randomly selected latent vectors where visually they are close to normal, to see more details, Fig. 11 shows corresponding *Q-Q plot* (Wikipedia contributors, 2026g) which shows more details about normality of these latent vectors.

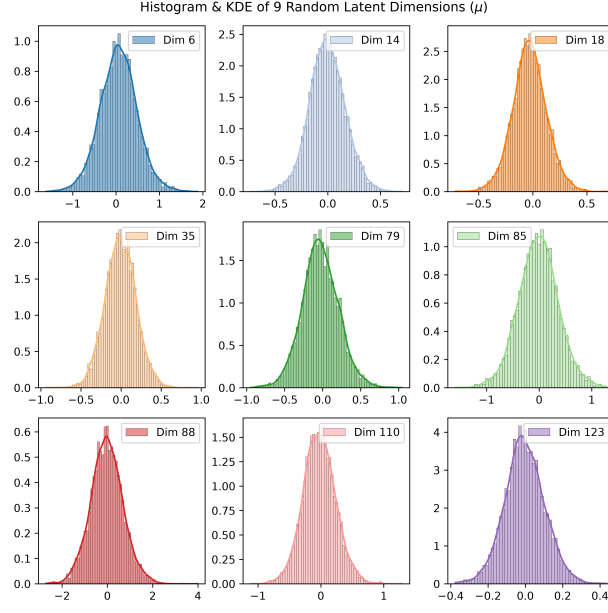


Figure 10

Sample Latent Vectors Distributions.

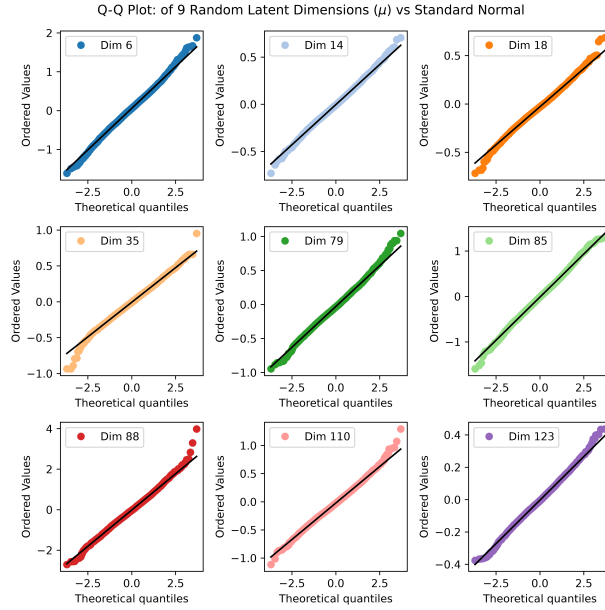


Figure 11

Sample Latent Vectors Q-Q Plot.

The Q-Q plot shows strong alignment with the normal distribution in the central region, but exhibits some departure at one end or both ends. This indicates that the latent

dimension has more extreme values or outliers than a standard normal distribution would predict. Fig. 12 shows latent vectors correlation matrix which is not perfect but very close to ideal case of Identity matrix.

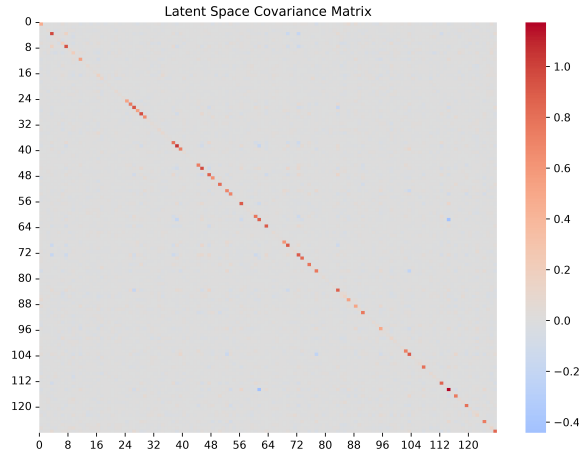


Figure 12

Latent Vectors Correlation Matrix.

Top-4 Images with Best and Worst Reconstruction

Fig. 9 shows that we have outliers in minimum and maximum range of total losses, and we can select top-4 images with worst (maximum) or top-4 images with best (minimum) total loss for visual investigation. The result are shown in Fig. 13.

- Interestingly the top 4 images with best reconstruction loss are not a good quality images, even one of them is totally invalid (almost all black) images, detecting such an images for removal from dataset and training can be an interesting future work.
- While reconstructed images in VAE is not sharp, but they are generally smooth (sometimes too smooth), but in the worst case here, the image is not smooth at all especially on the face looks like chessboard pattern (In the lower-right image the face is not recognizable at all)

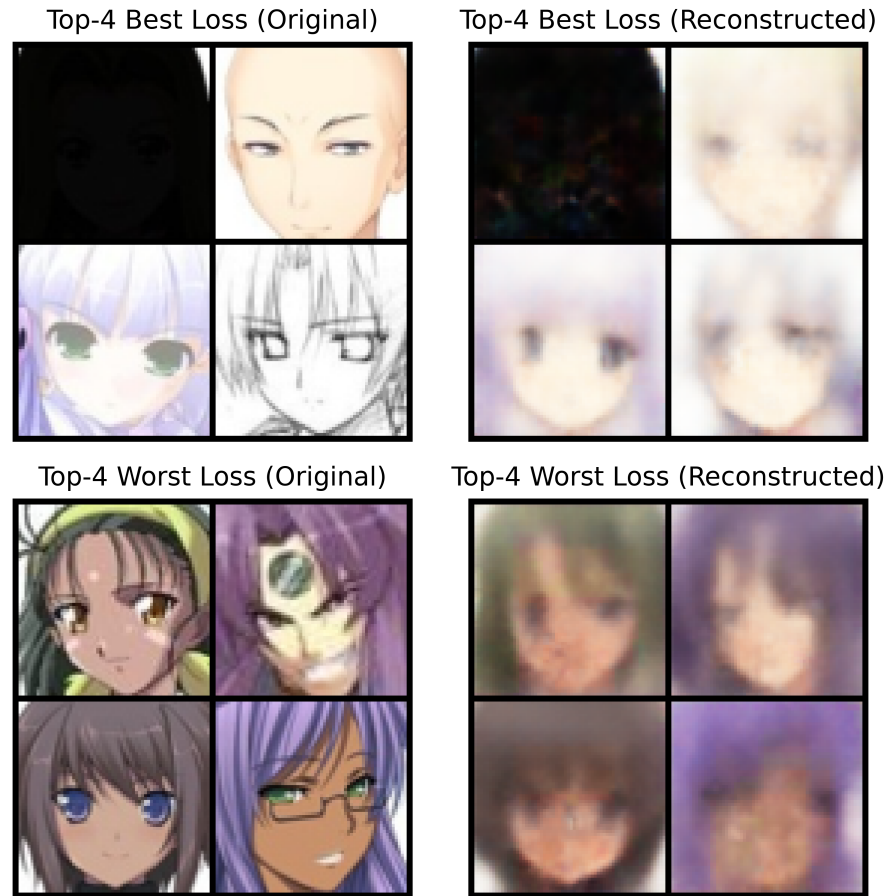


Figure 13

Top-4 Images with Best and Worst Reconstruction.

Sample Images with their Reconstructed versions

In previous section (Fig. 13) we saw the images with worst and best reconstruction, here in Fig. 14 we show sample random images with their reconstructed versions both with best model and the last-epoch model (that has minimum KLD-loss but shows the failure mode of VAE: mode averaging (blurriness)).

Sample Generated Images From Noise Vector

Finally in Fig. 15 we can see sample generated images with both best model and the last-epoch model (has mode averaging issue).

- The above images are generated anime face from a same normal random latent

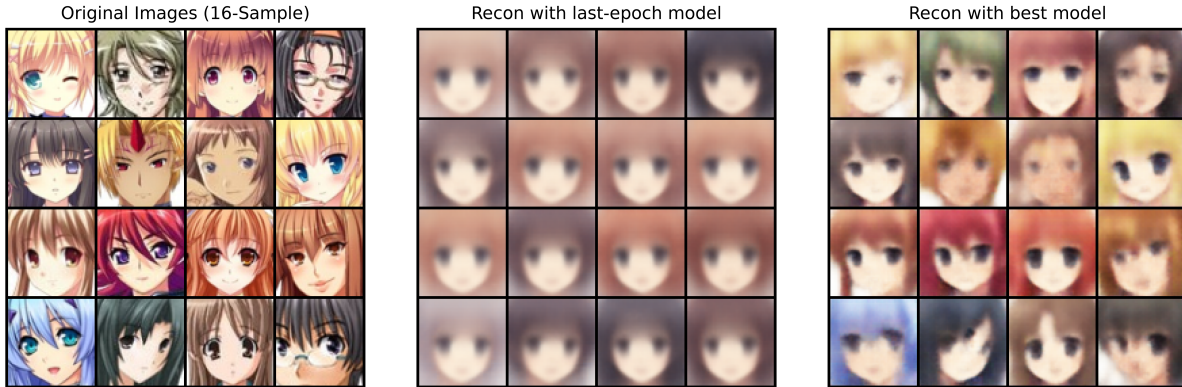


Figure 14

Sample Random Images with Their Reconstruction.

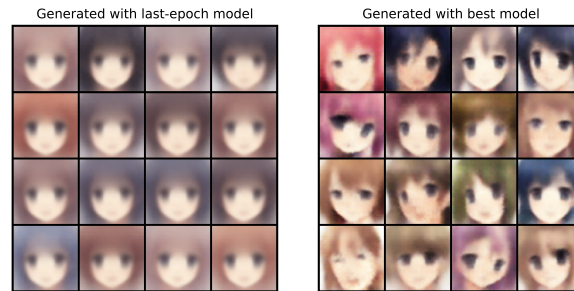


Figure 15

Sample Generated Images From Noise Vector.

vectors

- Last-epoch model has min KLD-loss but fails to generate diverse sample (has mode averaging (blurriness) issue)
- Best model has best validation loss with limited KLD-loss and generated a diverse set of images
- Overall best model generate smooth colorful images, but it does not have sharpness or details in more advanced models like DCGAN (Radford et al., 2016) or StyleGAN (Karras et al., 2019).

Discussion of Generated Outputs: Quality, Variability, and Limitations

The generated samples illustrated in Fig. 15 highlight the performance dynamics, visual quality, and inherent structural boundaries of our VAE architecture:

- **Quality:** The best model (epoch 27) reliably captures high-level global structures of anime faces, including skin tone variations, hair color diversity, eye positioning, and general head orientation. The generated images exhibit smooth transitions without harsh grid artifacts or severe spatial distortions.
- **Variability:** Sampling directly from the standard normal latent distribution $\mathcal{N}(0, \mathbf{I})$ yields diverse output faces with varied hair styles, facial expressions. This confirms that the encoder successfully mapped the training distribution onto a continuous, smooth, and well-behaved latent space without severe collapse.
- **Limitations:** The generated outputs lack sharp high-frequency details. Fine features such as distinct eye pupils, individual hair strands, mouth outlines, and sharp line art appear soft or slightly smudged. This blurriness is a fundamental limitation of pixel-wise reconstruction losses (e.g., BCE/MSE combined with KLD regularization), which tend to average over plausible pixel configurations rather than committing to sharp boundaries.

Ethical Considerations and Responsible Use

Generative modeling of human and stylized faces introduces key ethical considerations regarding data representation, creative ownership, and potential misuse:

- **Dataset Representation:** The dataset comprises web-scraped images (Chao, 2019), which even for anime inherit specific stylistic biases compared to global anime art. Therefore the model can be biased and not representative of global anime styles.
- **Artist Ownership and Copyright:** Anime art datasets often aggregate illustrations without explicit permission or licensing from the original creators.

Automatically generating anime might be unfair to original creators, and violate copyright and creative ownership.

- **Deepfake and Identity Misuse Risks:** Although stylized anime faces pose lower risks of real-person identity fraud compared to photorealistic human face generators, similar architectures or improved versions can be repurposed to generate synthetic media for impersonation.

Limitations and Future Improvements

While our model successfully implemented Variational Auto-Encoders (VAE) on image synthesis, several architectural and data processing limitations persist:

- **Architectural Limitations:** The simple 4-layer convolutional topology restricts spatial resolution to 64×64 . This is not really deep CNN for analyzing and capturing details in image.
- **Dataset Quality Control:** As identified in [Top-4 Images with Best and Worst Reconstruction](#)-section, the dataset contains low-quality, corrupt, or near-black outliers that negatively impact model generalization and loss calculations.

To address these limitations, future work should explore the following realistic improvements:

1. **Improve Model Architecture:** Use deeper networks with better architecture (e.g. with idea from ResNet18 (He et al., [2015](#))), or use more advanced VAE like NAVE (Vahdat & Kautz, [2021](#)) to support higher-resolution generation (128×128 or 256×256).
2. **Automated Data Cleaning Pipeline:** Implementing an automated filtering mechanism (e.g., density-based anomaly detection or brightness/contrast thresholds) to automatically remove invalid or corrupted images prior to split and training.

References

- Bank, D., Koenigstein, N., & Giryes, R. (2020). Autoencoders. *CoRR*, *abs/2003.05991*.
<https://arxiv.org/abs/2003.05991>
- Chao, B. (2019). *Anime face dataset: A collection of high-quality anime faces*.
<https://github.com/bchao1/Anime-Face-Dataset>
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition.
<https://arxiv.org/abs/1512.03385>
- Karras, T., Laine, S., & Aila, T. (2019). A style-based generator architecture for generative adversarial networks. <https://arxiv.org/abs/1812.04948>
- Kingma, D. P., & Welling, M. (2019). An introduction to variational autoencoders. *CoRR*, *abs/1906.02691*. <http://arxiv.org/abs/1906.02691>
- Kingma, D. P., & Welling, M. (2022). Auto-encoding variational bayes.
<https://arxiv.org/abs/1312.6114>
- PyTorch Foundation. (2026a). Adam Optimizer documentation [Accessed: 2026-08-22].
<https://docs.pytorch.org/docs/2.13/generated/torch.optim.Adam.html>
- PyTorch Foundation. (2026b). ExponentialLR Scheduler documentation [Accessed: 2026-08-22]. https://docs.pytorch.org/docs/2.13/generated/torch.optim.lr_scheduler.ExponentialLR.html
- Radford, A., Metz, L., & Chintala, S. (2016). Unsupervised representation learning with deep convolutional generative adversarial networks.
<https://arxiv.org/abs/1511.06434>
- Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. <https://arxiv.org/abs/1409.1556>
- splcher. (2019). *Anime Face Dataset [dataset]*. Kaggle.
<https://www.kaggle.com/datasets/splcher/animefacedataset>
- Vahdat, A., & Kautz, J. (2021). Nvae: A deep hierarchical variational autoencoder.
<https://arxiv.org/abs/2007.03898>

Wikipedia contributors. (2026a). Activation function — Wikipedia, the free encyclopedia [Online; accessed 30-August-2026]. [https:](https://en.wikipedia.org/w/index.php?title=Activation_function&oldid=1362367872)

[//en.wikipedia.org/w/index.php?title=Activation_function&oldid=1362367872](https://en.wikipedia.org/w/index.php?title=Activation_function&oldid=1362367872)

Wikipedia contributors. (2026b). Convolutional neural network — Wikipedia, the free encyclopedia [Online; accessed 31-August-2026]. https://en.wikipedia.org/w/index.php?title=Convolutional_neural_network&oldid=1368942578

Wikipedia contributors. (2026c). Cross-entropy — Wikipedia, the free encyclopedia [Online; accessed 30-August-2026]. <https://en.wikipedia.org/w/index.php?title=Cross-entropy&oldid=1371248242>

Wikipedia contributors. (2026d). Evidence lower bound — Wikipedia, the free encyclopedia [Online; accessed 31-August-2026]. [https:](https://en.wikipedia.org/w/index.php?title=Evidence_lower_bound&oldid=1360656798)
[//en.wikipedia.org/w/index.php?title=Evidence_lower_bound&oldid=1360656798](https://en.wikipedia.org/w/index.php?title=Evidence_lower_bound&oldid=1360656798)

Wikipedia contributors. (2026e). Kullback–leibler divergence — Wikipedia, the free encyclopedia [Online; accessed 31-August-2026]. https://en.wikipedia.org/w/index.php?title=Kullback%E2%80%93Leibler_divergence&oldid=1364249363

Wikipedia contributors. (2026f). Latent space — Wikipedia, the free encyclopedia [Online; accessed 31-August-2026]. https://en.wikipedia.org/w/index.php?title=Latent_space&oldid=1369640556

Wikipedia contributors. (2026g). Q–Q plot — Wikipedia, the free encyclopedia [Online; accessed 31-August-2026]. [https:](https://en.wikipedia.org/w/index.php?title=Q%E2%80%93Q_plot&oldid=1340026062)
[//en.wikipedia.org/w/index.php?title=Q%E2%80%93Q_plot&oldid=1340026062](https://en.wikipedia.org/w/index.php?title=Q%E2%80%93Q_plot&oldid=1340026062)